COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 20, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Acelajado, John Vince D. Pangilinan Jr., Bienvenido	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA NO. 14.1 **Algorithm Complexity**

1. PROCEDURE OUTPUT

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Best Case	Worst Case	Analysis
Algo 1: Mege Sort	O(n log n)	O(n log n)	In Merge Sort it will always divide the list into its two equal halves which sorts each half then merges them back together and it's already sorted. Because of what merge sort does it will take about the same number of the steps no matter how the data is arranged so the best and worst cases are the same which is O(n log n) **For different values of n examples:** n = 100: about 664 steps n = 1,000: about 9,966 steps n = 10,000: about 132,877 steps When the n will become 10 times bigger its steps will only grow a little more than 10 times so the merge sort will stays fast even for its large data.
Algo 2: Quick Sort	O(n log n)	O(n^2)	In Quick Sort it picks pivot based on the pivot it will split the list into smaller and larger values. The best case is O(n log n) which will happen when the pivot will land near the middle of the array every time so the list of it will be divided evenly. The worst case is O(n^2) which will happen when the pivot is always the smallest or the largest value while picking its first element of an already sorted list which the array of list is divided very unevenly. This sorts in place and it is usually fast and not stable. **For different values of n examples:** n = 100: about 664 steps (best) vs. 4,950 steps (worst) n = 1,000: about 9,966 steps (best) vs. 499,500 steps (worst) n = 10,000: about 132,877 steps (best) vs. 49,995,000 steps (worst) When the n becomes 10 times bigger. The best case will grow little more than 10 times and the worst case grows 100 times. Which means the quick sort will become very fast with a good pivot and will become very slow on large data with a bad one.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

	Algo 1: Merge Sort	**Algo 2: Quick Sort**
Best Case	100: Best case : 21.236 us 1000: Best case : 196.886 us 10000: Best case : 1718.73 us	100: Best case : 12.5895 us 1000: Best case : 49.536 us 10000: Best case : 661.133 us
Worst Case	100: Worst case: 13.8375 us 1000: Worst case: 133.518 us 10000: Worst case: 1593.2 us	100: Worst case: 43.4399 us 1000: Worst case: 4568.96 us 10000: Worst case: 439240 us

Analysis:

Acelajado

- Based on that result, Merge Sort is consistent as the best and worst cases are very close at 10000 elements with 1718.73 µs and 1593.2 µs respectively, which is consistent with the predicted O(n log n) growth. In the worst case, Quick Sort is 439240 µs, which is approximately 275 times slower than Merge Sort, due to the possibility of making bad pivot choices that result in the worst case of $O(n^2)$. This indicates that Merge Sort is a safer algorithm to use when dealing with data that is large or not sorted so it does not rely on the way data is arranged. The best algorithm is dependent on the type of data that we are going to sort, as Quick Sort is more effective when the data is random, and the pivot is well selected.

Pangilinan

- After analyzing the code, I noticed that quick sort is significantly faster than merge sort when inputs are ideal, but merge sort is far more reliable because its performance stays consistent while quick sort suffers extreme slowdowns on bad inputs.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.
For the theoretical analysis, the deliverable must be:

- Pseudocode Analysis
- Time and Space Complexities

For the empirical analysis, the deliverable must be the filled-out table and your analysis.

THEORETICAL ANALYSIS

Pseudocode:

Linear Search

```
BEGIN
   INPUT array, target

   FOR i = 0 TO N - 1 DO
      IF array[i] = target THEN
         RETURN i
      END IF
   END FOR

   RETURN -1
END
```

Binary Search

```
BEGIN
   INPUT array, target
   low = 0
   high = N - 1

   WHILE low <= high DO
      mid = (low + high) DIV 2

      IF array[mid] = target THEN
         RETURN mid
      ELSE IF array[mid] < target THEN
         low = mid + 1
      ELSE
         high = mid - 1
      END IF
   END WHILE

   RETURN -1
END
```

Pseudocode Analysis:

Acelajado

- Linear Search will look at each item in the list until it reaches the number. If the number is first, it will be fast O(1). If it's not there, it'll check everything which beccomes O(n). Binary Search is O(log n) which performs a search on a sorted list by discarding half of it at a time. Their space requirement is O(1) in both, since they have only a few variables.

Pangilinan

- Reading the two pseudocodes, Linear Search is the simple one. It goes through the list one item at a time from the start, so the more items there are, the more checks it may need. That is why its worst case is O(n). Binary Search is smarter because it checks the middle item and throws away the half that cannot hold the target. Cutting the list in half again and again means it needs very few steps even for a large list, which gives O(log n). The catch is that Binary Search only works if the list is already sorted, while Linear Search works on any list. Both use only a few extra variables, so both have O(1) space.

Time and Space Complexities

	Best	**Average**	**Worst**	**Space**
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)

EMPERICAL ANALYSIS		
Input Size (N)	**Elapsed Time for Binary Search (Worst-Case)**	**Elapsed Time for Linear Search (Worst-Case)**
1,000	`0.000017`	`0.000779`
10,000	`0.000028`	`0.007788`
100,000	`0.000029`	`0.077694`
Code:		

```cpp
#include <iostream>
#include <iomanip>
#include <chrono>

template <typename T>
int linearSearch(const T data[], int arrSize, T dataFind){
    int i = 0;
    while(i < arrSize){
        if(dataFind == data[i]){
            return i;
        }
        i++;
    }
    return -1;
}

template <typename T>
int binarySearch(const T data[], int arrSize, T dataFind){
    int low = 0;
    int up = arrSize - 1;
    while(low <= up){
        int mid = low + (up - low) / 2;
        if(dataFind == data[mid]){
            return mid;
        }
        else if(dataFind < data[mid]){
            up = mid - 1;
        }
        else{
            low = mid + 1;
        }
    }
    return -1;
}

int main(){
    const int sizes[] = {1000, 10000, 100000};
    const int runs = 1000;
    volatile int sink = 0;

    std::cout << std::fixed << std::setprecision(6);
    std::cout << std::left << std::setw(12) << "N" << std::setw(34) << "Binary Search Worst-Case (ms)" << std::setw(34) << "Linear Search Worst-Case (ms)" << "\n";

    for(int s = 0; s < 3; s++){
        int n = sizes[s];
        int* data = new int[n];
        for(int i = 0; i < n; i++){
            data[i] = i * 2;
        }
        int target = -1;

        auto b0 = std::chrono::high_resolution_clock::now();
        for(int r = 0; r < runs; r++){
            sink = binarySearch(data, n, target);
        }
        auto b1 = std::chrono::high_resolution_clock::now();

        auto l0 = std::chrono::high_resolution_clock::now();
        for(int r = 0; r < runs; r++){
            sink = linearSearch(data, n, target);
        }
        auto l1 = std::chrono::high_resolution_clock::now();

        double binMs = std::chrono::duration<double, std::milli>(b1 - b0).count() / runs;
        double linMs = std::chrono::duration<double, std::milli>(l1 - l0).count() / runs;

        std::cout << std::left << std::setw(12) << n << std::setw(34) << binMs << std::setw(34) << linMs << "\n";
        delete[] data;
    }
    return 0;
}
```

Output:

N	Binary Search Worst-Case (ms)	Linear Search Worst-Case (ms)
1000	0.000017	0.000779
10000	0.000028	0.007788
100000	0.000029	0.077694

Analysis:

Acelajado

- Linear Search was found to be slower in our results as N increased. When N became 10 times bigger, the time also became about 10 times bigger 0.000779 to 0.007788 to 0.077694 ms. Even with N ten times larger, Binary Search's time stayed just a bit worse 0.000017 to 0.000029 ms. Linear Search was thousands of times slower than Binary Search when both algorithms were implemented with 100,000 iterations.

Pangilinan

- As N grew from 1,000 to 100,000, Linear Search took longer and longer, rising from 0.000779 ms to 0.077694 ms, which is about 100 times more for 100 times more data. Binary Search barely changed, only moving from 0.000017 ms to 0.000029 ms. At 100,000 items, Linear Search was around 2,600 times slower than Binary Search. This shows how much better O(log n) is than O(n) once the data gets big.

Conclusion

Acelajado

- In this lab, I learned that the speed of an algorithm is dependent on its increase as the data increases. Merge Sort remains fast and stable, while Quick Sort is fast with a good pivot, but is very slow with a poor pivot. Linear Search slows down as the data increases in size while Binary Search remains efficient but is only applicable to sorted data. As our experiment shows the theory is correct, the best algorithm is therefore dependent on the type of data.

Pangilinan

- From this activity, I learned that we should not judge an algorithm by how fast it runs on a small list, but by how its time grows when the data grows. Merge Sort is steady because its time barely depends on how the data is arranged, while Quick Sort can be very fast or very slow depending on its pivot. For searching, Binary Search is much faster than Linear Search on large data, but it needs a sorted list first.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar Correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Use only for grammar

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.